INFRASTRUCTURE MANAGEMENT TOOL

Infra Portal

A self-service portal for requesting servers on Proxmox VE and AWS

Muthu · November 2026 · github.com/Muthujar/infra-portal

THE PROBLEM

Getting a server today means a ticket and a wait

- Developers raise a ticket and wait for the DevOps team.
- Servers are built by hand, so they differ.
- There is no clear record of who approved what.
- Forgotten test servers keep using capacity.

Goal: a catalog, an approval step and automation, like VMware vRealize Automation (Aria).

THE SOLUTION

Four steps from request to ready server

1 · DEVELOPER

Request

Picks a server, environment and lease.

2 · API

Check

Validates the input and team quota.

3 · TEAM LEAD

Approve

Approves it. PROD needs an admin. Terraform is generated.

4 · WORKER

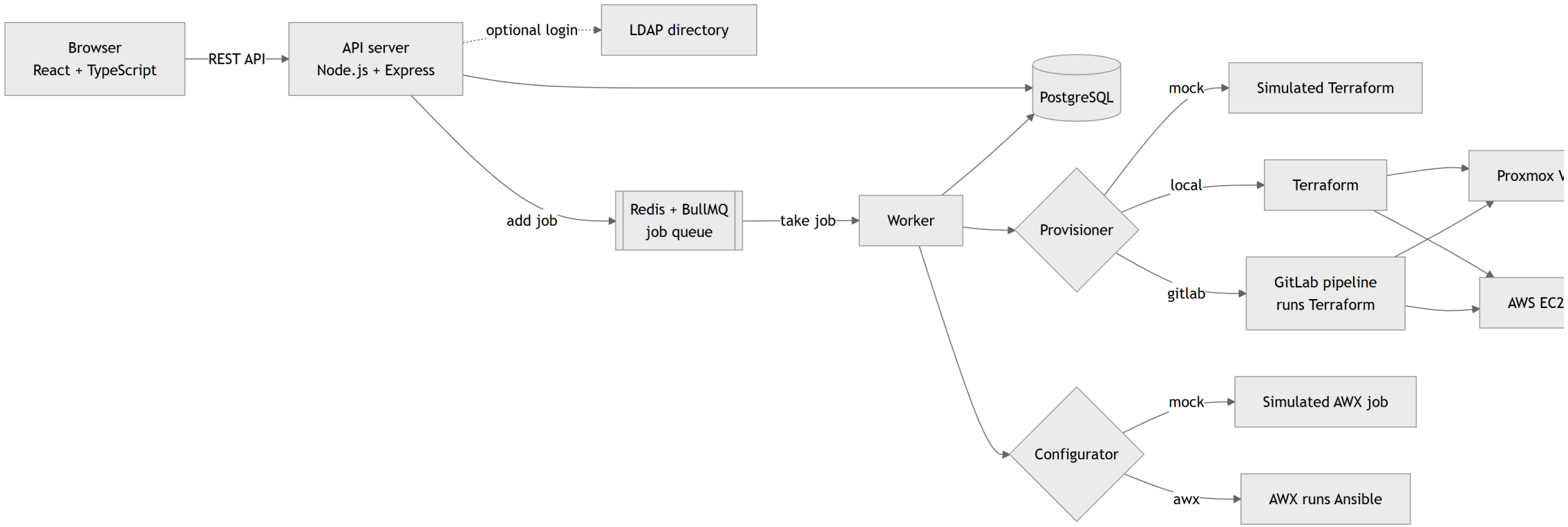
Build

Runs Terraform, then configures the server.

Every change is in the audit log. Servers are deleted when their lease ends.

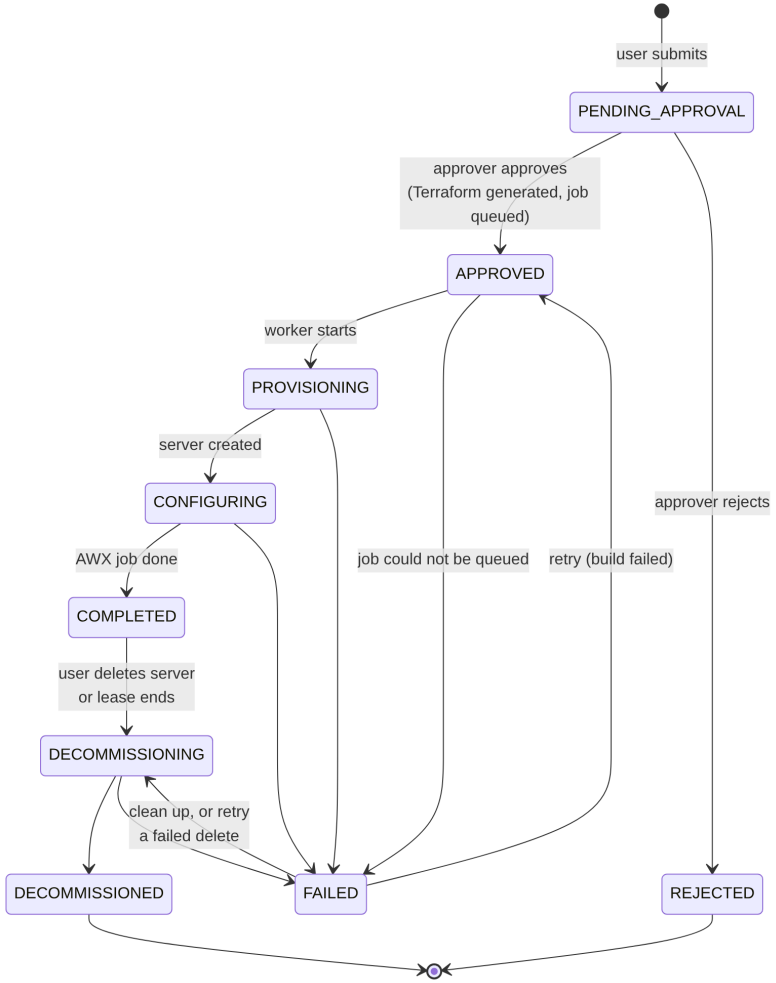
ARCHITECTURE

A queue and worker handle the slow work



REQUEST LIFE CYCLE

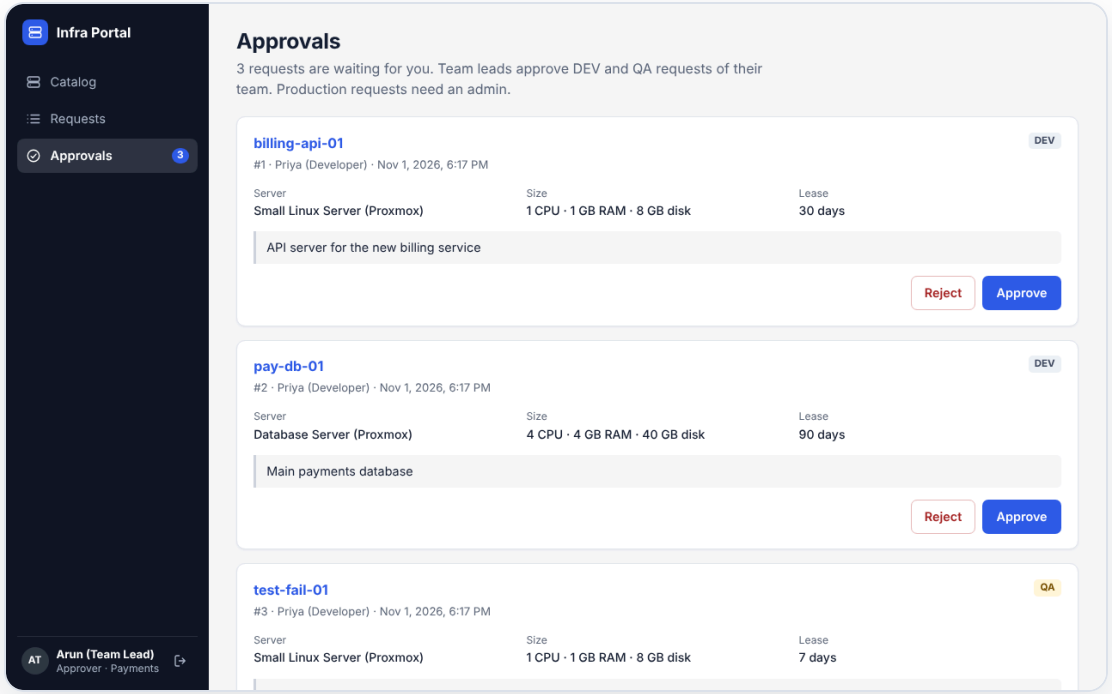
One state machine controls every status change



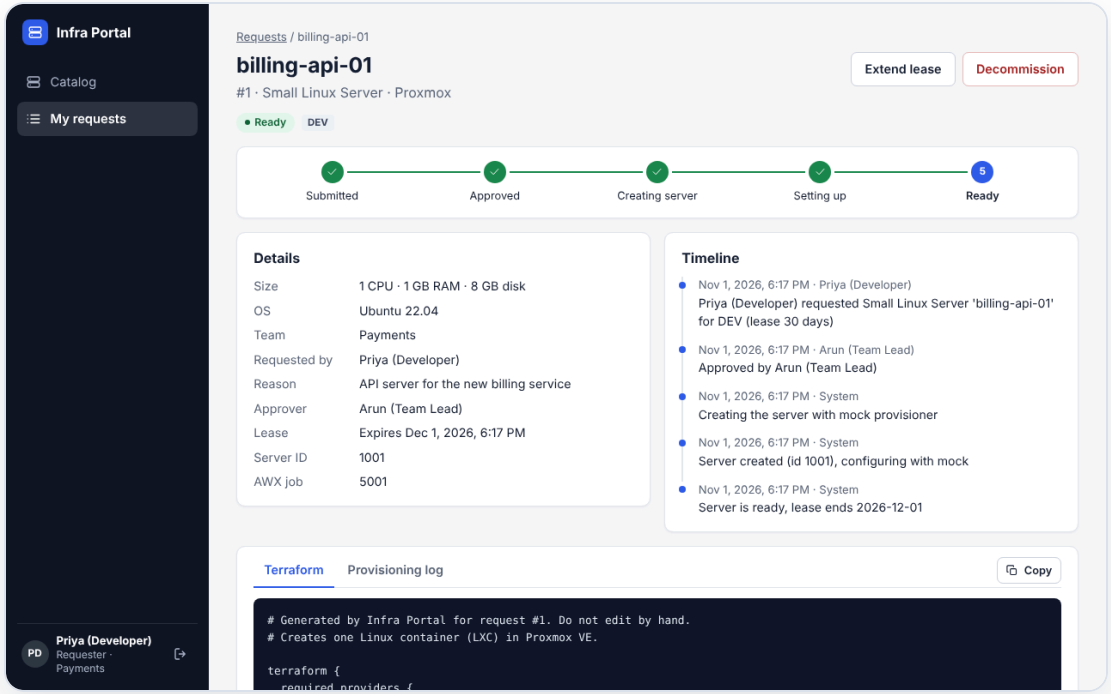
- Only allowed moves are accepted.
- Two people cannot act on a request at once.
- Every change is written to the audit log.

DEMO

From approval to a ready server



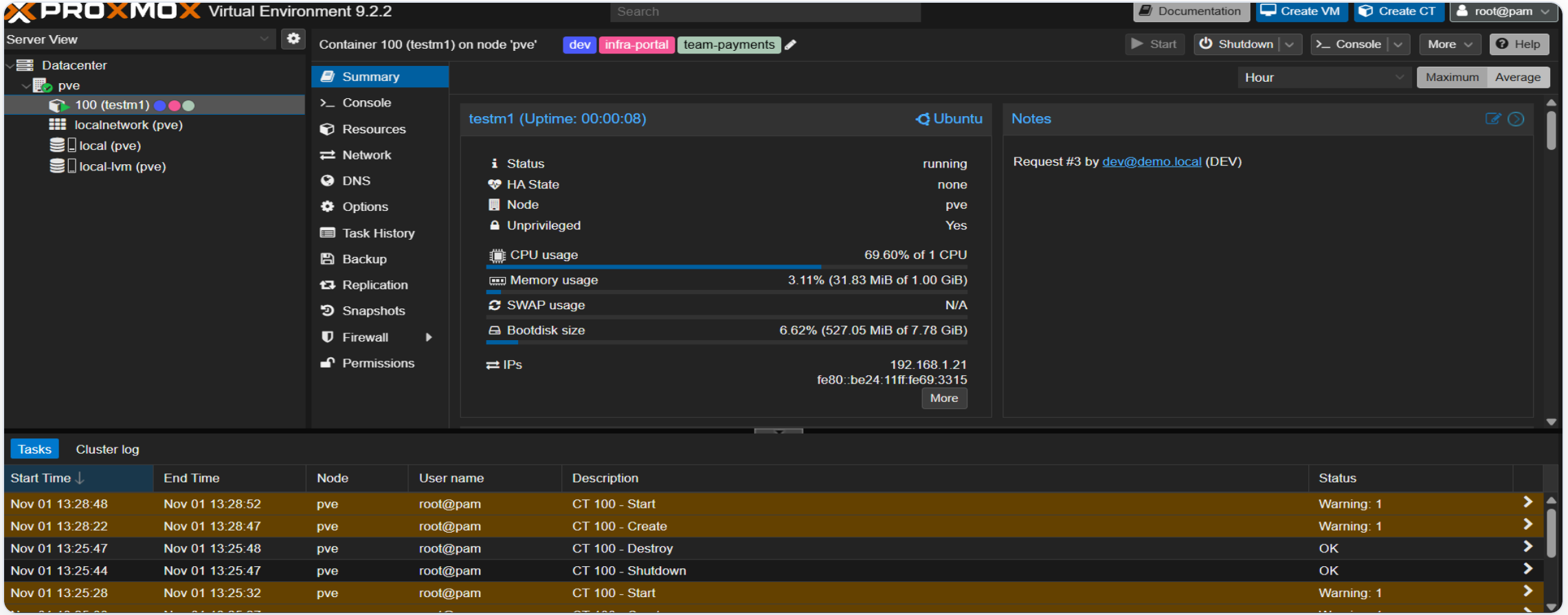
Team lead approves



Live status, timeline and Terraform

TESTED FOR REAL

A real container on Proxmox VE 9.2



Created by Terraform in about 40 seconds, then deleted on decommission.

CONTROLS

Who can do what, and how much

Roles

Requester, approver, admin. PROD needs an admin.

Quotas

CPU, RAM and disk limits for each team.

Audit log

Who did what, and when.

Leases

Expired servers are deleted automatically.

Secrets

Never saved in Terraform files or the database.

LDAP login

Directory groups set role and team.

INCLUDED

What is built and tested

PART	STATUS
Portal, workflow, roles, quotas, leases, audit log	Done
Terraform generation for Proxmox and AWS	Done
Provisioning on real Proxmox VE	Tested
LDAP login	Tested
44 automated tests	Passing
CI for the backend, frontend, Terraform and Docker	Passing

IF THIS MOVES FORWARD

How it would connect to a company environment

AREA	HOW IT WOULD BE DONE
GitLab CI	Point the built connector at the company GitLab, with a runner inside the network
AWX	Point the built connector at the company AWX, and add a dynamic inventory
AWS	Add a company AWS account and turn on apply
Company login	Connect the company directory or identity provider, and reuse the role mapping
State and secrets	Move Terraform state to shared storage, and secrets to a vault

The demo uses mock provisioning and configuration, so these need a full run in a company environment.